Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 3: Multi-threading
Spring Semester 2026

Assigned on: **04.03.2026**

Due by: **(Wednesday Exercise) 09.03.2026**
(Friday Exercise) 11.03.2026

Overview

This week's assignment is about multi-threaded programs in Java.

- Fetch the new assignment from Gitlab by running:

  ```
  git fetch upstream
  git merge upstream/main
  ```

  You should now see the new assignment appear in your local directory.
- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

Task 1 – Parallel Counting

Description: In this exercise we will implement a `Counter` that allows counting the number of times a given event occurred. We will start with the following interface of the `Counter`:

```
public interface Counter {
    public void increment();
    public int value();
}
```

As can be seen, the counter has only two methods – `increment` that increases the `Counter` value by one and `value` that returns the current value of the `Counter`. We will use the `Counter` to monitor the progress of executing multiple threads that perform the following loop:

```
for (int i = 0; i < numIterations; i++) {
  ... // perform some work

  counter.increment();
}
```

For simplicity, in this assignment no actual work will be performed and each thread will only increment the counter `numIterations` times. The implementation of the above loop is already provided to you in the `NativeThreadCounter` class. In what follows we will study several different approaches that implement the `Counter` such that it can be safely used by multiple threads.

JUnit Tests: We provide JUnit tests to check the basic functionality of your solution.

Notice: Java libraries not included in the assignment project are not allowed. Do not rename any of the provided methods in the assignment (you can create additional classes and methods).

Tasks

A) To start with, implement a sequential version of the `Counter` in `SequentialCounter` class that does not use any synchronization. That is, the counter simply increments an integer value by one. We already provide code in `taskASequential` method that runs a single thread that increments the counter. Inspect the code and understand how it works. Verify that the `SequentialCounter` works properly when used with a single thread (the test `testSequentialCounter` should pass). Now run the code in `taskAParallel` which creates several threads that all try to increment the counter at the same time. Notice how the expected value of counter at the end of the execution is not what we would expect. Discuss why this is the case.

B) To fix this issue, implement a different thread safe version of the `Counter` in `SynchronizedCounter`. In this version use the standard primitive type **`int`** but synchronize the access to the variable by inserting **`synchronized`** blocks. Run the code in `taskB`.

C) Whenever the `Counter` is incremented, keep track which thread performed the increment (you can print out the thread-id to the console). Can you see a pattern in how the threads are scheduled? Discuss what might be the reason for this behaviour.

D) Implement a `FairThreadCounter` that ensures that different threads increment the `Counter` in an round-robin fashion. In round-robin scheduling the threads perform the increments in circular order. That is, two threads with ids 1 and 2 would increment the value in the following order $1, 2, 1, 2, 1, 2,$ etc. You should implement the scheduling using the `wait` and `notify` methods. Can you think of an implementation that does not use `wait` and `notify` methods? Extend your implemenation to work with arbitrary number of threads (instead of only 2) that increment the counter in round-robin fashion.

E) Implement a thread safe version of the `Counter` in `AtomicCounter`. In this version we will use an implementation of the **`int`** primitive value, called `AtomicInteger`*, that can be safely used from multiple threads. Run the code in `taskE` that should now produce correct results even with multiple threads. Note: You are allowed to import `java.util.concurrent.atomic.AtomicInteger` for this task.

F) Compare the `AtomicCounter` and `SynchronizedCounter` implementations by measuring which one is faster. Observe the differences in the CPU load between the two versions. Can you explain what is the cause of different performance characteristics?

G) Implement a thread that measures the execution progress. That is, create a thread that observes the values of the `Counter` during the execution and prints them to the console. Make sure that the thread is properly terminated once all the work is done.

*https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/AtomicInteger.html

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:
  ```
  git add .  # this command adds all changes in the git repo
  ```
- Commit your changes. In your commit message, make sure to include the [Submission] tag.
  ```
  git commit -m "[Submission] ..."
  ```
- Push your changes!
  ```
  git push
  ```

To submit your code using IntelliJ, follow the instructions below:

- Press the "Commit" button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press "Commit and Push"

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!

Troubleshooting

Git Issues
Missing repo:
In rare cases, you might get an error like this:

```
Can't connect to any URI
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git
(https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git:
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git/info/refs?
service=git-receive-pack not found)
```

It means that no repository was created for your username yet. If that is the case, please contact your teaching assistant. The teaching assistant will be able to create the repository location for you. After that, you should be able to submit your work by following the instructions above.

Issues fetching from upstream:
If you are unable to fetch upstream, or get issues related to SSH vs HTTPS when merging from upstream, you can change your upstream URL using the following command:

```
git remote set-url upstream https://gitlab.inf.ethz.ch/course-pprog26/pprog-26-assignments.git
```

IntelliJ / Java Issues
If you have trouble running the Java programs in IntelliJ, check the following:

- You have marked the `src` directory as a "Source Root". Right click on the folder `src`, select "Mark Directory As >Source Root".
- You have installed an updated JDK. We recommend `openjdk-25`. You can check this under "File >Project Structure >SDK"